

26 Juni 2026 – 30 Juni 2026

# Dokumentasi

## Technical Assessment - Cloud Engineer

Oleh:

**Randi Palguna Artayasa**  
**5025231020**



[github.com/randiPalguna/pve-lab](https://github.com/randiPalguna/pve-lab)

ITS Nabu Recruitment 2026

## Daftar Isi

|                                                                       |    |
|-----------------------------------------------------------------------|----|
| 1. Instalasi Proxmox VE Lokal .....                                   | 3  |
| 1.1. Konfigurasi Virtual Machine (VM) untuk PVE pada VirtualBox ..... | 3  |
| 1.2. Konfigurasi Instalasi PVE .....                                  | 4  |
| 1.3. Konfigurasi Jaringan pada PVE Node .....                         | 6  |
| 1.4. Konfigurasi SSH Authentication pada PVE Node .....               | 7  |
| 2. Manajemen Template & Container .....                               | 8  |
| 3. Implementasi Layanan .....                                         | 11 |
| 4. Analisis Urgensi .....                                             | 14 |
| 5. Penggunaan AI .....                                                | 15 |
| 5.1. Penggunaan AI untuk Mempelajari Konsep .....                     | 15 |
| 5.2. Penggunaan AI untuk Menangani Masalah .....                      | 15 |
| 5.3. Penggunaan AI untuk Menghasilkan Kode .....                      | 15 |

## Daftar Gambar

|                                                                               |    |
|-------------------------------------------------------------------------------|----|
| Gambar 1 Konfigurasi image dan OS VirtualBox .....                            | 3  |
| Gambar 2 Aktifkan fitur nested virtualization .....                           | 3  |
| Gambar 3 Gunakan Bridged Adapter dan Promiscuous Mode “Allow All” .....       | 4  |
| Gambar 4 Contoh konfigurasi PVE Node Network .....                            | 4  |
| Gambar 5 Hapus ISO installer pada setting storage VM PVE .....                | 5  |
| Gambar 6 Web GUI PVE dapat diakses menggunakan IP address PVE Node .....      | 5  |
| Gambar 7 Topologi jaringan yang ingin dibangun dalam PVE Node .....           | 6  |
| Gambar 8 Topologi jaringan untuk container yang akan di-provision .....       | 8  |
| Gambar 9 Hasil provisioning container menggunakan Terraform .....             | 10 |
| Gambar 10 Diagram eksekusi kontrol Ansible melalui SSH .....                  | 11 |
| Gambar 11 Client melakukan request ke IP PVE node dengan interface nic0 ..... | 13 |
| Gambar 12 Hasil uji coba load balancing round robin .....                     | 13 |

## Daftar Kode

|                                                                |    |
|----------------------------------------------------------------|----|
| Kode 1 Konfigurasi /etc/network/interfaces pada PVE node ..... | 7  |
| Kode 2 Konfigurasi awal main.tf .....                          | 9  |
| Kode 3 Konfigurasi variables.tf .....                          | 9  |
| Kode 4 Konfigurasi terraform.tfvars .....                      | 10 |
| Kode 5 Konfigurasi inventory.ini .....                         | 12 |
| Kode 6 Konfigurasi ansible.cfg .....                           | 12 |

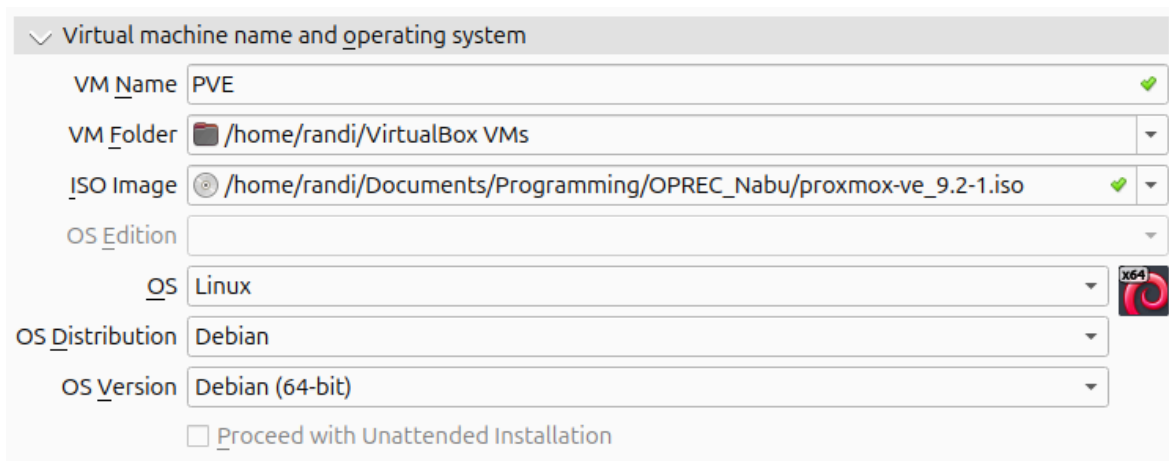
# 1. Instalasi Proxmox VE Lokal

Kita dapat menginstall file ISO Proxmox VE (PVE) pada internet. Kemudian kita menggunakan software Oracle VirtualBox untuk dapat menjalankan PVE di dalam host kita. Berikut adalah langkah-langkah untuk membuat VM PVE di dalam VirtualBox hingga mengkonfigurasi lingkungan PVE agar siap untuk melakukan provision container dan deployment service di dalamnya.

## 1.1. Konfigurasi Virtual Machine (VM) untuk PVE pada VirtualBox

Setelah mengunduh software VirtualBox dan file ISO PVE, kita akan membuat VM PVE dengan langkah-langkah sebagai berikut:

1. Kita mengatur Linux sebagai OS dan Debian 64 Bit sebagai version OS-nya pada konfigurasi VM VirtualBox.



Gambar 1: Konfigurasi image dan OS VirtualBox

2. Disini kita mengalokasikan 10800 MB memory, 9 CPU, dan 80.00 GB storage (sesuai dengan kapasitas host yang dapat dialokasikan).
3. Setelah itu kita perlu mengaktifkan fitur Nested VT-x/AMD-V pada VirtualBox. Hal ini dikarenakan kita menjalankan PVE sebagai VM dan PVE ingin membuat container atau VM lagi di dalamnya, maka dibutuhkan nested virtualization.

Features ☐ PAE/NX  
☒ Nested VT-x/AMD-V

Gambar 2: Aktifkan fitur nested virtualization

4. Kemudian pada konfigurasi network VM, kita perlu menggunakan Bridged Adapter agar web GUI PVE dapat diakses dari jaringan LAN yang dimiliki host (laptop yang menjalankan VM PVE di dalam VirtualBox) tanpa perlu melakukan port forwarding.  
Selain itu, Promiscuous Mode perlu diubah menjadi “Allow All” agar semua unicast frame (tanpa peduli source dan destination MAC addressnya) dapat keluar masuk secara bebas pada Bridged Adapter VirtualBox sehingga host manapun dalam LAN dapat mengakses container Proxmox VE yang memiliki MAC address tersendiri.

**Network**

Adapter 1 | Adapter 2 | Adapter 3 | Adapter 4

☒ Enable Network Adapter

Attached to: Bridged Adapter

Name: wlp0s20f3

Adapter Type: Intel PRO/1000 MT Desktop (82540EM)

Promiscuous Mode: Allow All

MAC Address: 080027386A78

☒ Virtual Cable Connected

Gambar 3: Gunakan Bridged Adapter dan Promiscuous Mode “Allow All”

## 1.2. Konfigurasi Instalasi PVE

Kita melakukan proses instalasi PVE menggunakan GUI agar lebih mudah. Berikut adalah langkah-langkah instalasi PVE hingga PVE dapat diakses di web browser host (laptop yang digunakan untuk menjalankan VM PVE):

1. Isi target disk, location, time zone, keyboard layout, root password, email sesuai dengan keinginan.
2. • Selanjutnya, IP Address (CIDR) dan Gateway akan otomatis terisi sendiri dimana jika kita menggunakan WiFi (atau ethernet) dari ISP, PVE akan otomatis mendapatkan IP-nya dari DHCP. Untuk saat ini, IP Address (CIDR) dan Gateway akan mengikuti IP yang diperoleh dari DHCP. Kita dapat menggantinya jika kita menggunakan LAN yang berbeda di kemudian hari.
  - Untuk Hostname (FQDN), kita bebas menamainya (namun tidak menggunakan reserved hostname) seperti “pve.lan”.
  - DNS Server kita atur menjadi “1.1.1.1” agar memudahkan koneksi ke Internet jika menggunakan domain.

Management Interface: nic0 - 08:00:27:38:6a:78 (e1000)

Hostname (FQDN): pve.lan

IP Address (CIDR): 192.168.110.92 / 24

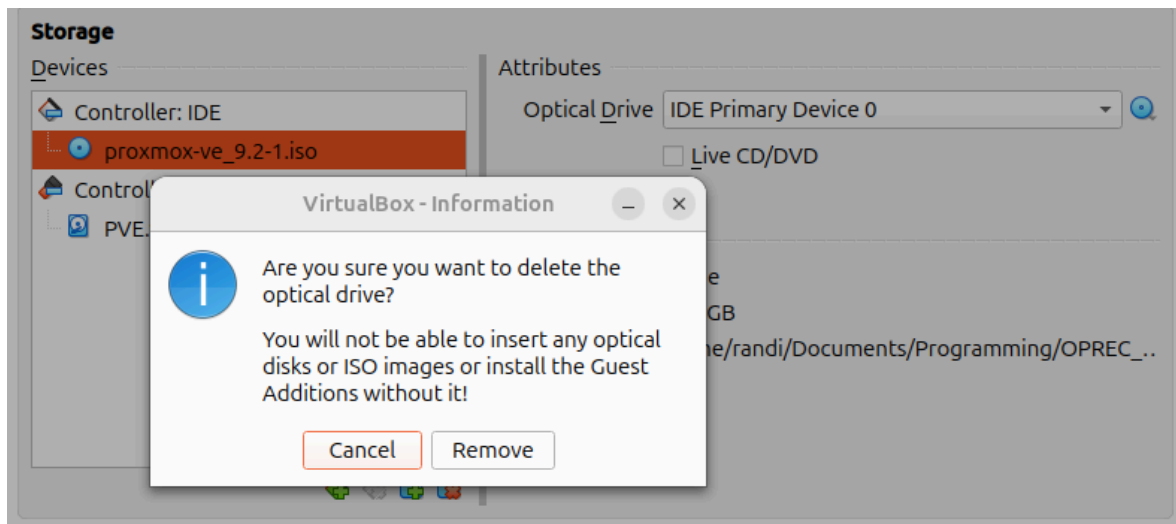
Gateway: 192.168.110.1

DNS Server: 1.1.1.1

☒ Pin network interface names Options

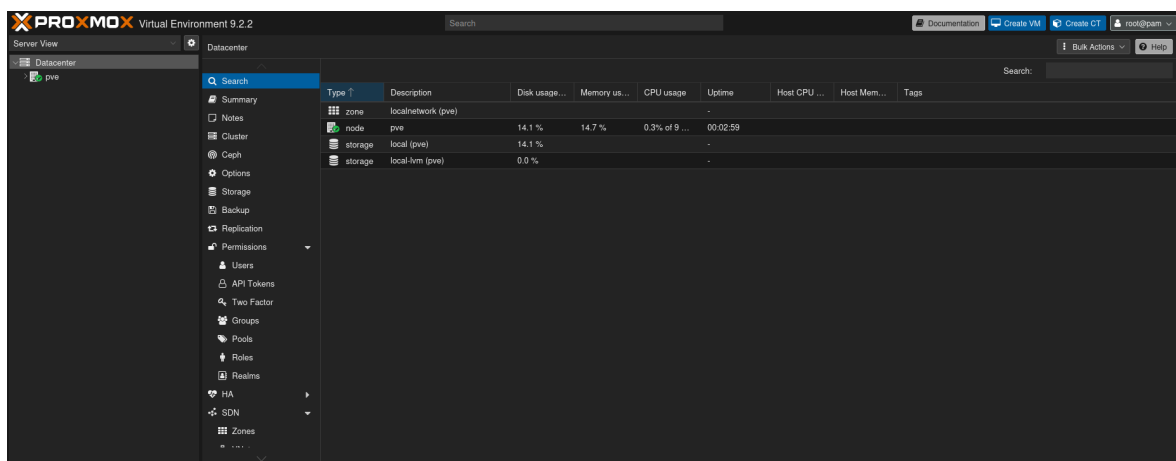
Gambar 4: Contoh konfigurasi PVE Node Network

3. Lakukan proses instalasi dan reboot. Setelah selesai, kita dapat shutdown VM PVE lalu kita perlu menghapus image installer ISO pada VM-nya di dalam setting VM PVE VirtualBox.



Gambar 5: Hapus ISO installer pada setting storage VM PVE

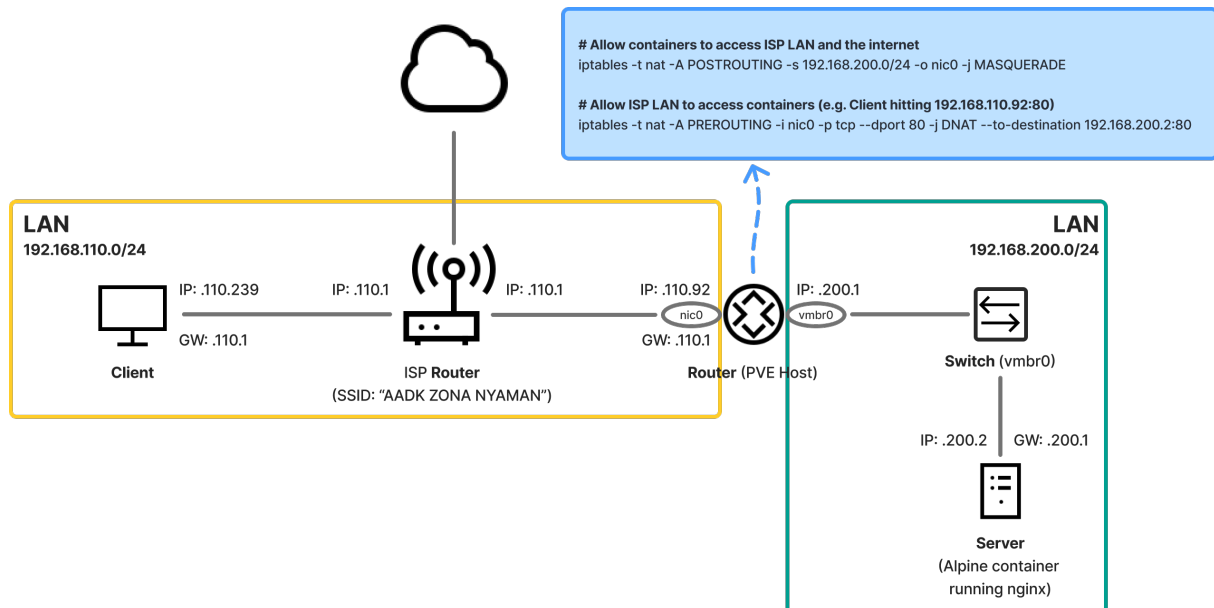
4. Terakhir, jalankan kembali VM PVE dan akses web GUI PVE menggunakan IP address PVE yang diberikan DHCP yakni <http://192.168.110.62> pada host (laptop) kita. Lalu masukan username “root” dan password yang sesuai.



Gambar 6: Web GUI PVE dapat diakses menggunakan IP address PVE Node

### 1.3. Konfigurasi Jaringan pada PVE Node

Berikut adalah gambaran topologi jaringan yang akan kita konfigurasi nantinya pada lingkungan PVE.



Gambar 7: Topologi jaringan yang ingin dibangun dalam PVE Node

Terdapat 2 Local Area Network (LAN) di dalam topologi jaringan tersebut. LAN yang pertama adalah LAN yang disediakan dari Internet Service Provider (ISP), dimana end hosts terhubung melalui medium Wi-Fi. LAN yang kedua adalah private subnet yang dibuat di dalam PVE untuk keperluan jaringan container. Selanjutnya, terdapat Network Address Translation (NAT) dan IP forwarding di dalam PVE Node router. NAT dibutuhkan agar container dapat mengakses LAN ISP dan juga internet dengan menggunakan IP dari nic0. Sedangkan IP forwarding dibutuhkan agar end hosts yang terhubung dalam LAN ISP dapat mengakses service yang disediakan oleh container (misalnya nginx web server).

#### Catatan:

Dibuatnya private subnet untuk container dikarenakan Wi-Fi dari access point router ISP memiliki batasan untuk hanya memperbolehkan device yang terhubung mengirimkan frame ke router dengan source MAC address dari device tersebut, tidak boleh source MAC address yang lain. Ini artinya ketika container mengirimkan unicast frame ke ISP router dengan source MAC Address-nya tersendiri, unicast frame tersebut akan di-drop oleh ISP router. Dalam kasus ini, frame yang dapat diterima oleh ISP router adalah frame dengan MAC address dari laptop yang menjalankan VM PVE ini.

Untuk mengimplementasikan topologi jaringan tersebut, kita perlu masuk kedalam shell node pve kita dan mengatur konfigurasi network interface port dari PVE node (router). Berikut adalah langkah-langkahnya:

1. Atur konfigurasi interface port pada PVE node, yakni mengatur IP address dan gateway dari interface nic0 dan vmbro0 sesuai dengan gambar topologi jaringan diatas. Pada shell node pve, edit file /etc/network/interfaces dengan konfigurasi yang ditunjukkan pada Kode 1.

---

#### **/etc/network/interfaces**

---

```
auto lo
iface lo inet loopback

auto nic0
iface nic0 inet static
    address 192.168.110.92/24
    gateway 192.168.110.1

source /etc/network/interfaces.d/*

auto vmbr0
iface vmbr0 inet static
    address 192.168.200.1/24
    bridge-ports none
    bridge-stp off
    bridge-fd 0
    post-up    echo 1 > /proc/sys/net/ipv4/ip_forward

    ## containers to ISP LAN and internet ##
    post-up    iptables -t nat -A POSTROUTING -s 192.168.200.0/24 -o nic0 -j
MASQUERADE
    post-down iptables -t nat -D POSTROUTING -s 192.168.200.0/24 -o nic0 -j
MASQUERADE

    ## ISP LAN with PROTO tcp, PORT 80 -> containers with DST PORT 80, DST IP
192.168.200.2 ##
    post-up    iptables -t nat -A PREROUTING -i nic0 -p tcp --dport 80 -j DNAT
--to-destination 192.168.200.2:80
    post-down iptables -t nat -D PREROUTING -i nic0 -p tcp --dport 80 -j DNAT
--to-destination 192.168.200.2:80
```

---

Kode 1: Konfigurasi /etc/network/interfaces pada PVE node

2. Selanjutnya, eksekusi perintah `ifreload -a` untuk memperbarui konfigurasi interface PVE node.

#### **Catatan:**

Terdapat script yang digunakan untuk mengotomatisasi konfigurasi interface pve node diatas. Script tersebut dapat dilihat pada link berikut:

- `set_pve_network.sh`:  
[github.com/randiPalguna/pve-lab/blob/main/script/set\\_pve\\_network.sh](https://github.com/randiPalguna/pve-lab/blob/main/script/set_pve_network.sh)
- `set_pve_ip_forwarding.sh`:  
[github.com/randiPalguna/pve-lab/blob/main/script/set\\_pve\\_ip\\_forwarding.sh](https://github.com/randiPalguna/pve-lab/blob/main/script/set_pve_ip_forwarding.sh)

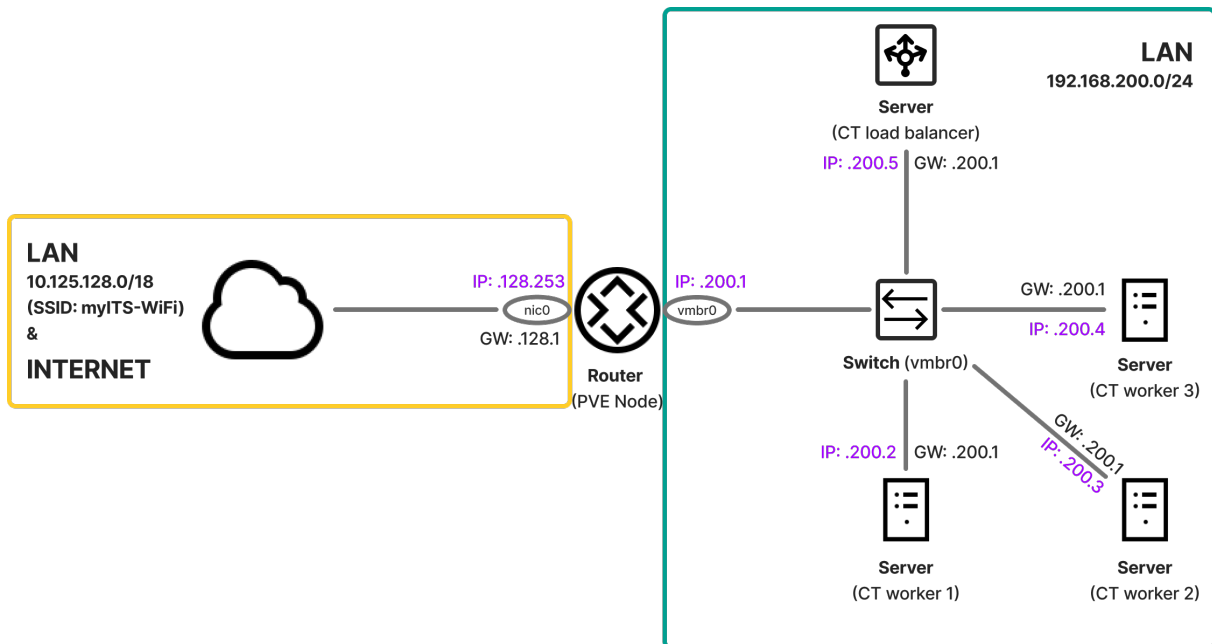
## **1.4. Konfigurasi SSH Authentication pada PVE Node**

Untuk memudahkan penggunaan tool Ansible nantinya, PVE Node perlu memiliki public-key dari host yang akan mengontrol PVE Node dan containernya, pada kasus ini host tersebut adalah laptop yang menjalankan PVE pada VirtualBox. Berikut adalah cara untuk mengkonfigurasi SSH Authentication:

1. Pada host yang menjadi control node, buatlah ssh key (private dan public key) dengan cara  
`ssh-keygen -t ed25519`
2. Setelah membuat ssh key, kita dapat membagikannya ke PVE Node dengan cara  
`ssh-copy-id -i ~/.ssh/id_ed25519.pub root@<PVE Node address>`

## 2. Manajemen Template & Container

Berikut adalah Gambar 8 yang menunjukkan gambaran container-container yang akan kita provision nantinya. Terdapat 3 container sebagai worker yang akan menyediakan layanan nginx web server, dan 1 container sebagai load\_balancer yang akan mendistribusikan requests service ke ketiga worker tersebut. Adapun spesifikasi dari masing-masing container adalah 1 CPU, 512 MB memory, 1 GB disk, dan OS Debian 13. Untuk penerapan jaringannya, masing-masing container memiliki DNS 1.1.1.1 dengan IP dan Gateway yang dapat dilihat pada Gambar 8.



Gambar 8: Topologi jaringan untuk container yang akan di-provision

### Catatan:

Sebelumnya pada **1. Instalasi Proxmox VE Lokal**, kita menggunakan jaringan LAN ISP dengan network address **192.168.110.0/24**. Kali ini pada **2. Manajemen Template & Container**, kita menggunakan jaringan myITS-WiFi dengan network address **10.125.128.0/18**. Dengan adanya perubahan ini, interface nic0 pada PVE node perlu diganti dengan network address jaringan myITS-WiFi di dalam file `/etc/network/interfaces`.

Untuk melakukan provision container di dalam PVE, kita dapat menggunakan tool Terraform di dalam host yang menjalankan PVE pada VirtualBox (host tersebut dapat disebut dengan control node). Terraform memungkinkan kita untuk membuat, mengedit, ataupun menghapus container-container PVE dengan cara mendeklarasikannya dalam bentuk kode konfigurasi (Infrastructure as Code) yang bersifat deklaratif. Untuk memulai membuat container-container menggunakan Terraform, kita perlu mengunduh Terraform disini <https://developer.hashicorp.com/terraform/install>. Setelah mengunduhnya, berikut adalah cara menggunakan Terraform untuk melakukan provisioning container template dan containernya:

1. Buatlah suatu direktori baru dan buatlah file bernama `main.tf` di dalamnya dengan konfigurasi awal seperti Kode 2.



---

**main.tf**

---

```
terraform {
  required_version = "1.15.6"
  required_providers {
    proxmox = {
      version = "0.11.0"
      source  = "bpg/proxmox"
    }
  }
}

provider "proxmox" {
  endpoint      = var.endpoint
  username      = var.pve_username
  password      = var.pve_password
  insecure      = true
  random_vm_ids = true
}
```

---

Kode 2: Konfigurasi awal main.tf

2. Karena kita menggunakan variables dalam melakukan assignment value di dalam main.tf, maka diperlukan file variables.tf dan terraform.tfvars. File variables.tf adalah tempat untuk mendeklarasikan variable-variable, sedangkan file terraform.tfvars adalah tempat untuk memberikan value terhadap beberapa variable yang telah didefinisikan, contohnya adalah string pve\_password yang bersifat sensitif. Berikut adalah konfigurasi pada kedua file tersebut.

---

**variables.tf**

---

```
variable "endpoint" {
  type = string
}

variable "pve_node_name" {
  type      = string
  default = "pve"
}

variable "pve_username" {
  type      = string
  sensitive = true
}

variable "pve_password" {
  type      = string
  sensitive = true
}
...
```

---

Kode 3: Konfigurasi variables.tf

#### terraform.tfvars

```
endpoint      = "https://10.125.128.253:8006/"
pve_username  = "root@pam"
pve_password  = "pve_pass"
...
```

#### Kode 4: Konfigurasi terraform.tfvars

3. Jalankan perintah `terraform init` pada direktori dengan file-file `main.tf`, `variables.tf`, dan `terraform.tfvars` di dalamnya.
4. Setelah proses inialisasi selesai, selanjutnya kita akan mendeklarasikan resource-resource yang nantinya akan dibuat pada PVE. Resource-resource tersebut diantaranya adalah
  - `lxc image` (resource `"proxmox_download_file"`),
  - `container template` (resource `"proxmox_virtual_environment_container"`)
  - `container asli` (resource `"proxmox_virtual_environment_container"`)

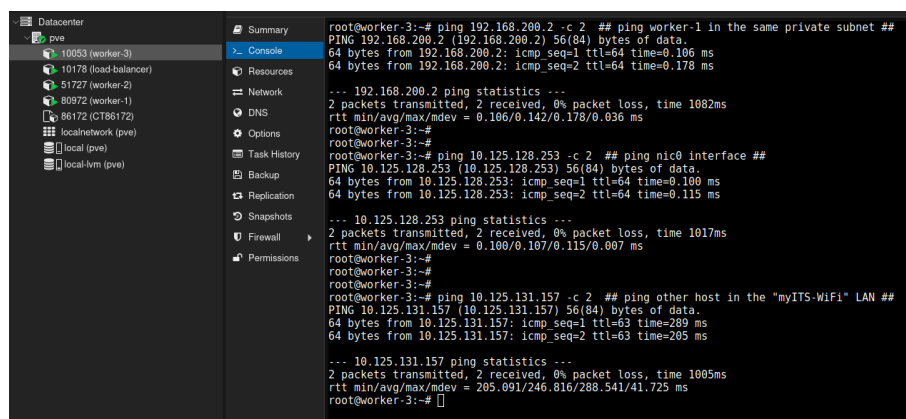
Container asli dapat dibuat dengan melakukan cloning dari container template untuk mengurangi pengulangan penulisan di dalam konfigurasi. Selain itu, untuk keperluan tool Ansible, container perlu memiliki ssh public-key dari control node. Hal tersebut secara otomatis dapat Terraform lakukan saat melakukan provision create, yakni dengan mendeklarasikan blok `user_account` di konfigurasi `main.tf` dan memberikan path public-key control node pada field `keys` di dalam blok tersebut.

#### Catatan:

- Anda dapat melihat konfigurasi deklarasi untuk masing-masing resource pada link berikut: [github.com/randiPalguna/pve-lab/blob/main/terraform/main.tf](https://github.com/randiPalguna/pve-lab/blob/main/terraform/main.tf).
- Referensi konfigurasi provider `bpg/proxmox` dapat dilihat pada link: [registry.terraform.io/providers/bpg/proxmox/0.110.0/docs](https://registry.terraform.io/providers/bpg/proxmox/0.110.0/docs)

5. Setelah berhasil membuat dan mengkonfigurasi file-file Terraform, kita dapat menjalankan perintah `terraform fmt` dan `terraform validate` untuk memperbaiki format dan kesalahan penulisan.
6. Selanjutnya kita dapat melakukan provisioning container menggunakan terraform dengan menjalankan perintah `terraform apply -parallelism=1` dan ketik “yes” ketika diminta prompt. Kita menggunakan option `-parallelism=1` untuk menghindari CT lock error, namun membuat provisioning tidak berjalan secara parallel atau memakan waktu lebih lama.

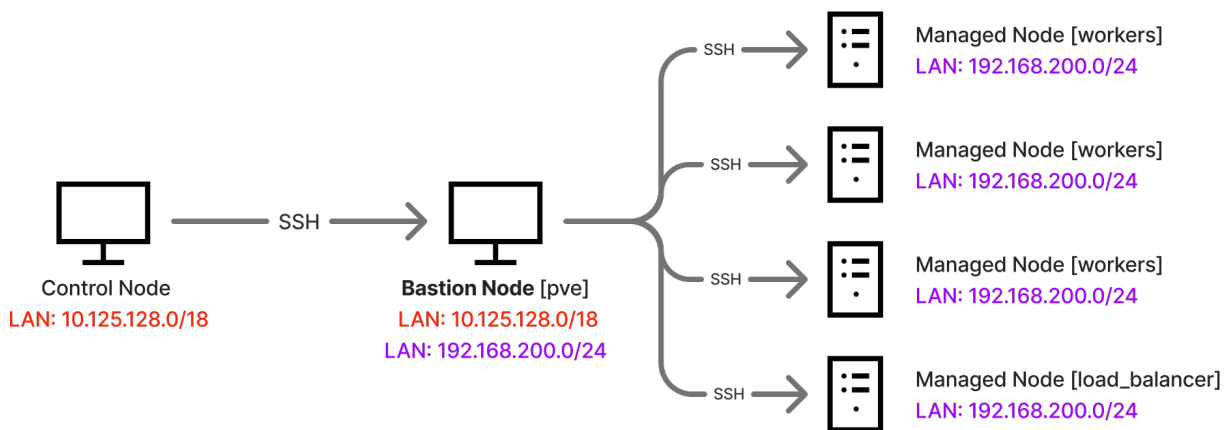
Berikut adalah Gambar 9 yang menunjukkan hasil provisioning container menggunakan Terraform.



Gambar 9: Hasil provisioning container menggunakan Terraform

### 3. Implementasi Layanan

Selanjutnya kita akan mengimplementasikan nginx web server pada workers container dan nginx load balancing pada load balancer container. Untuk mengimplementasikan layanan-layanan tersebut, kita dapat menggunakan tool Ansible untuk mengotomatisasikannya. Dengan Ansible, kita mengkonfigurasi seluruh container yang ada (dapat dikatakan sebagai “managed nodes”) dengan konfigurasi layanan yang didefinisikan pada file `playbook.yaml`. Dalam kasus ini, eksekusi ansible playbook berasal dari host yang menjalankan PVE pada VirtualBox (dapat dikatakan sebagai “control node”). Namun control node tidak dapat mengakses managed nodes secara langsung karena perbedaan subnet jaringan, maka dari itu diperlukan node yang dapat dijadikan sebagai perantara atau penengah (dapat dikatakan sebagai “bastion node”). Dalam kasus ini, bastion node tersebut adalah PVE Node. Berikut adalah Gambar 10 tentang bagaimana control node dapat mengakses managed nodesnya.



Gambar 10: Diagram eksekusi kontrol Ansible melalui SSH

Berikut adalah langkah-langkah untuk menggunakan Ansible dalam mengimplementasikan layanan nginx pada managed nodesnya:

1. Buatlah direktori baru dan di dalamnya buatlah python environment dengan menjalankan command `python3 -m venv .venv`
2. Setelah itu, unduh Ansible dengan menjalankan command `pip install ansible`
3. Lalu buatlah file bernama `inventory.ini` dan deklarasikan managed nodes dan bastion node didalamnya. Kita dapat mengakses managed nodes dari control node dengan kode baris `ansible_ssh_common_args=-o ProxyJump=root@10.125.128.253` pada `inventory.ini`. Berikut Kode 5 adalah konfigurasi `inventory.ini`

---

**inventory.ini**

---

```
[pve]
10.125.128.253

[workers]
192.168.200.2
192.168.200.3
192.168.200.4

[load_balancer]
192.168.200.5

[pve:vars]
ansible_user=root

[workers:vars]
ansible_user=root
ansible_ssh_common_args=-o ProxyJump=root@10.125.128.253

[load_balancer:vars]
ansible_user=root
ansible_ssh_common_args=-o ProxyJump=root@10.125.128.253
```

---

Kode 5: Konfigurasi inventory.ini

4. Selain itu, kita juga perlu membuat file bernama `ansible.cfg` untuk mengatur behavior dari Ansible seperti `known_host` ssh checking dan python interpreter warning message. Berikut konfigurasi `ansible.cfg`

---

**ansible.cfg**

---

```
[defaults]
inventory = inventory.ini
host_key_checking = False
interpreter_python = auto_silent
```

---

Kode 6: Konfigurasi ansible.cfg

5. Akhirnya kita membuat file `playbook.yaml` dan mendeklarasikan “play” untuk masing-masing node beserta tasks-nya. Misalnya adalah kita mendeklarasikan suatu play khusus untuk node `workers` dimana terdapat beberapa task didalamnya seperti:

- Mengunduh package `nginx`
- Mengkonfigurasi `nginx` file
- Membuat konten `html` yang menarik
- Melakukan restart pada service `nginx`

Pada node `load_balancer`, kita mendeklarasikan task seperti berikut:

- Mengunduh package `nginx`
- Mengkonfigurasi `nginx` file untuk melakukan load balancing dengan round robin
- Melakukan restart pada service `nginx`

Pada node `pve`, kita mendeklarasikan task seperti berikut:

- Menghapus semua iptables dengan chain `PREROUTING`
- Menambahkan IP dan port forwarding untuk mengarahkan requests ke IP `load_balancer`.

6. Terakhir, jalankan perintah `ansible-playbook playbook.yaml` untuk mengeksekusi Ansible.

**Catatan:**

Anda dapat melihat konfigurasi `playbook.yaml` pada link berikut:  
[github.com/randiPalguna/pve-lab/blob/main/ansible/playbook.yaml](https://github.com/randiPalguna/pve-lab/blob/main/ansible/playbook.yaml)

Berikut adalah hasil implementasi layanan menggunakan Ansible, dapat dilihat pada Gambar 11. Gambar tersebut membuktikan bahwa IP forwarding telah berhasil mengarahkan request ke node load\_balancer serta layanan nginx telah berhasil dihidupkan oleh Ansible.



Gambar 11: Client melakukan request ke IP PVE node dengan interface nic0

Untuk membuktikan bahwa load balancing berhasil diimplementasi, jalankan perintah:

```
curl -s 10.125.128.253 | grep "worker-"
```

```
randi@Pongo-725:~$ curl -s 10.125.128.253 | grep "worker-"
<title>Dashboard - worker-2</title>
<div class="info-row"><span class="label">Hostname:</span><span class="value">worker-2</span></div>
randi@Pongo-725:~$ curl -s 10.125.128.253 | grep "worker-"
<title>Dashboard - worker-3</title>
<div class="info-row"><span class="label">Hostname:</span><span class="value">worker-3</span></div>
randi@Pongo-725:~$ curl -s 10.125.128.253 | grep "worker-"
<title>Dashboard - worker-1</title>
<div class="info-row"><span class="label">Hostname:</span><span class="value">worker-1</span></div>
randi@Pongo-725:~$ curl -s 10.125.128.253 | grep "worker-"
<title>Dashboard - worker-2</title>
<div class="info-row"><span class="label">Hostname:</span><span class="value">worker-2</span></div>
randi@Pongo-725:~$ curl -s 10.125.128.253 | grep "worker-"
<title>Dashboard - worker-3</title>
<div class="info-row"><span class="label">Hostname:</span><span class="value">worker-3</span></div>
randi@Pongo-725:~$ curl -s 10.125.128.253 | grep "worker-"
<title>Dashboard - worker-1</title>
<div class="info-row"><span class="label">Hostname:</span><span class="value">worker-1</span></div>
randi@Pongo-725:~$ curl -s 10.125.128.253 | grep "worker-"
<title>Dashboard - worker-2</title>
<div class="info-row"><span class="label">Hostname:</span><span class="value">worker-2</span></div>
randi@Pongo-725:~$
```

Gambar 12: Hasil uji coba load balancing round robin

#### 4. Analisis Urgensi

ITS Nabu menawarkan sebuah platform *training cybersecurity* edukatif berbasis simulasi. Platform tersebut memiliki sesi *training* interaktif yang dapat meningkatkan pengalaman pengguna dalam mempelajari bidang ilmu *cybersecurity*. Untuk membangun platform tersebut, dibutuhkan *container* atau *virtual machine* yang akan dijadikan objek studi di dalam suatu kasus masalah pada sesi *training*-nya. Misalnya dalam suatu kasus masalah, suatu *virtual machine* telah disusupi sebuah *malware* dimana pengguna perlu mencari *malware* tersebut dan menganalisisnya. Ini membuktikan bahwa sesi *training* yang ada pada platform memerlukan beberapa *container* atau *virtual machine*, sehingga pemahaman dan keterampilan tentang Proxmox beserta manajemen *container* dan *template*-nya sangatlah esensial karena teknologi virtualisasi ini merupakan *tools* dan *resource* yang **vital** dalam platform ITS Nabu.

Dengan memiliki keterampilan yang baik tentang Proxmox beserta *container*-nya, tim developer dapat mengurangi kesalahan atau *error* dalam pengelolaan infrastruktur di ITS Nabu. Selain mengurangi kesalahan, dampak positif lainnya adalah komunikasi antar tim dalam melakukan hal yang bersifat teknis juga dapat lebih efektif dikarenakan konsep dan jargon-jargon yang ada dapat dipahami dengan makna yang sama. Maka dari itu, keterampilan yang baik dari masing-masing anggota tim akan mengantarkan tim developer yang lebih kompeten, namun tetap dibutuhkan sifat kerendahan hati dari masing-masing anggota untuk dapat belajar, berkolaborasi, dan saling memenuhi satu sama lainnya.

## 5. Penggunaan AI

Penulis menggunakan LLM atau AI (misalnya Claude dan Gemini) dalam mengerjakan tugas *technical assessment* ini, khususnya dalam hal pemahaman konsep yang mungkin sudah dilupakan atau belum diketahui. Selain itu, LLM juga digunakan dalam menangani masalah baik itu masalah jaringan maupun masalah sintaks dalam penulisan kode. Terakhir, LLM juga digunakan dalam menghasilkan penulisan kode khususnya kode untuk `playbook.yaml` Ansible. Berikut adalah detail mengenai penggunaan AI yang dilakukan dalam menyelesaikan tugas ini.

### 5.1. Penggunaan AI untuk Mempelajari Konsep

Dalam memahami konsep yang lupa ataupun belum diketahui, penulis bertanya ke LLM untuk mengisi ketidaktahuan tersebut. Contohnya mengenai konsep abstraksi jaringan logis pada PVE node. Selain itu juga konsep-konsep penggunaan tools Terraform dan Ansible juga digali menggunakan bantuan LLM. Namun penulis tetap menggunakan dokumentasi official yang tersedia di internet (seperti `bpg/proxmox registry`) dan tidak hanya bergantung dan mempercayai sepenuhnya kepada LLM.

### 5.2. Penggunaan AI untuk Menangani Masalah

Dalam pengerjaannya, penulis tidak luput dengan kesalahan seperti misalnya kesalahan sintaks pada penulisan kode atau bahkan kesalahan logika. Penulis menggunakan LLM untuk mengoreksi beberapa kesalahan yang muncul dan mempelajarinya.

### 5.3. Penggunaan AI untuk Menghasilkan Kode

Terakhir, penulis juga menggunakan LLM untuk menghasilkan sebuah kode, tepatnya pada kode `playbook.yaml`. Penulis masih belum familier dengan Ansible dan memiliki ketertarikan untuk menggunakannya dalam menyelesaikan tugas ini. Tidak terbatas dengan kurangnya pemahaman mengenai Ansible, penulis juga perlu memahami lebih lanjut mengenai konsep jaringan beserta alat untuk melakukan manajemen *container* seperti Proxmox dan Terraform.